# AUTONOMOUS AI WORKFORCE

7-Day API & Architecture Specification • 3 Developers • Basic + Advanced

**Purpose:** Lock the system architecture and API boundaries so Developer 1, Developer 2 and Developer 3 can work independently while producing one integrated product.

**Source basis:** The supplied architecture defines Event/Input → Perception → Orchestrator → Knowledge/Memory → Planner → Capability Manager → Guardrail/Security → Executive Twin or Specialist → Execution → Observation/Verification → Reflection/Re-plan → Output → Memory, plus a 3-member / 7-day target. ■filecite■turn3file1■L47-L87■

## 1. General Architecture

**USER → PERCEPTION → MODE SELECTOR → MASTER ORCHESTRATOR → MEMORY + PLANNER → CAPABILITY MANAGER → GUARDRAIL → TWIN / SPECIALIST → EXECUTION → OBSERVATION / VERIFICATION → OUTPUT / MEMORY.**

**Recovery:** Verification → Evidence → Reflection → Re-plan Gate → Planner → Retry → Verification. Core agents remain reusable as new capabilities are added. ■filecite■turn3file1■L83-L87■

## 2. API Ownership Map

| Service | Owner | Responsibility | Core API |
|---|---|---|---|
| Input / Perception | Dev 1 | Normalize voice/text/file input; intent, goals, constraints | POST /api/v1/perception |
| Task / Orchestrator | Dev 1 | Task creation, lifecycle, state, cancellation | POST /api/v1/tasks • GET /api/v1/tasks/{id} • POST /api/v1/tasks/{id}/cancel |
| Planner | Dev 1 | Steps, dependencies, criteria, stopping conditions | POST /api/v1/tasks/{id}/plan |
| Capability Manager | Dev 1 | Capability → worker/tool/model mapping | POST /api/v1/capabilities/resolve |
| Guardrail | Dev 1 | Allow/block/escalate + audit | POST /api/v1/security/check |
| Memory | Dev 3 | Obsidian retrieval, research validation, approved writes | POST /api/v1/memory/search • POST /api/v1/memory/write |
| Executive Twins | Dev 3 | Resolve/activate/recommend/delegate/review | POST /api/v1/twins/resolve • /twins/{id}/activate • /recommend • /delegate • /review |
| Specialists | Dev 3 | Registry, spawn, assignment, termination | GET /api/v1/agents • POST /api/v1/agents/spawn |
| Model Service | Dev 2 | Local model routing and inference | POST /api/v1/models/infer • GET /api/v1/models |
| Tool Executor | Dev 2 | Filesystem, terminal, Git, browser, Docker | POST /api/v1/tools/execute |
| Execution | Dev 2 | Controlled workspace actions and artifacts | POST /api/v1/executions • GET /api/v1/executions/{id} |
| Verification | Dev 1 + 2 | Objective tests, evidence, quality/security | POST /api/v1/verifications |
| Reflection / Re-plan | Dev 1 | Failure analysis + validated recovery | POST /api/v1/reflections • POST /api/v1/replans |
| Project / Walkthrough | Dev 3 | Project state, artifacts, final walkthrough | GET /api/v1/projects/{id} • GET /api/v1/projects/{id}/walkthrough |
| Events | Dev 1 + 3 | Activity + real-time task/workforce updates | GET /api/v1/events • WS /api/v1/ws/tasks/{id} |

## 3. Common Contracts — Shared by All 3 Developers

Internal implementation may differ. **Interface, field meaning and lifecycle semantics must not.**

### Task

| Field | Type | Purpose |
|---|---|---|
| id | string | Unique task ID |
| mode | basic \| advanced | Execution mode |
| input | object | Normalized request |
| requirements | string[] | Explicit requirements |
| constraints | string[] | Limits/policies |
| successCriteria | string[] | Verification requirements |
| status | pending \| running \| verifying \| completed \| failed \| cancelled | Lifecycle |
| planId / projectId | string? | Associated plan/project |
| result | object? | Final output |
| createdAt / updatedAt | timestamp | Audit timestamps |

### Plan / Agent / Execution / Verification

| Contract | Required fields |
|---|---|
| Plan | id, taskId, steps, dependencies, successCriteria, stoppingConditions, version |
| Agent | id, role, capability, model, tools, status, progress, taskId |
| Execution | id, agentId, action, workspace, toolCalls, modelCalls, artifacts, logs, status |
| Verification | id, taskId, checks, evidence[], passed, failures[], criteria, timestamp |
| Reflection | verificationId, evidence[], rootCause, confidence, proposedChanges |
| Replan | planVersion, changes[], evidenceRefs[], approved, reason |
| Memory | query, sources[], content, evidenceRefs[], approvalStatus, targetNote |

# 4. MEMBER 1 — CORE INTELLIGENCE & CONTROL

**Ownership:** Perception, Master Orchestrator, Planner, Capability Manager, Guardrail/Security, Execution Manager, Verification, Reflection/Re-plan and retry control. ■filecite■turn3file9■L500-L511■

## Basic Mode API path

POST /perception → POST /tasks → POST /capabilities/resolve → POST /security/check → model/agent execution → POST /verifications → result.

## Advanced Mode API path

POST /projects → POST /tasks → POST /memory/search → POST /tasks/{id}/plan → POST /capabilities/resolve → POST /security/check → workforce → execution → verification.

## Recovery API path

POST /reflections → POST /replans → security validation → POST /tasks/{id}/retry → execution → verification. Recommended MVP: maximum 3 recovery attempts.

## Member 1 — 7-Day Delivery

| Day | Deliverable |
|---|---|
| 1 | Backend skeleton, schemas, API conventions, logging |
| 2 | Perception + Master Orchestrator + task state |
| 3 | Planner + Capability Manager + Guardrail |
| 4 | Workforce routing contracts |
| 5 | Execution Manager + queues + timeouts + events |
| 6 | Verification + evidence + Reflection + Re-plan Gate |
| 7 | Full control-loop integration + stability |

# 5. MEMBER 2 — LOCAL AI, TOOLS & SOFTWARE DEVELOPMENT

**Ownership:** local model runtime, reasoning/coding/ASR integration, tool adapters, Docker, filesystem, terminal, Git, browser, Software Development and software verification.

## Basic Mode APIs

| API | Purpose | Example |
|---|---|---|
| POST /models/infer | Local reasoning/coding inference | structured model result |
| POST /models/transcribe | Audio → transcript | segments + transcript |
| POST /tools/execute | Approved tool action | result + evidence |
| GET /models | Model inventory/status | available models |
| GET /tools | Tool registry | available tools |

## Advanced Software Development APIs

| API | Purpose | Evidence |
|---|---|---|
| POST /executions | Start isolated workspace | executionId |
| POST /executions/{id}/files | Read/write project files | diff/hash |
| POST /executions/{id}/commands | Run approved commands | stdout/stderr/exitCode |
| POST /executions/{id}/tests | Run unit/integration/E2E | test report |
| POST /executions/{id}/security-scan | Security scan | security report |
| GET /executions/{id}/artifacts | Return generated artifacts | artifact metadata |

## Security boundary

The Tool Executor must not trust arbitrary model-generated commands. Approved action context comes from the control/security layer; the executor enforces workspace, command and permission restrictions and returns evidence.

## Member 2 — 7-Day Delivery

D1 GPU/model runtime → D2 model service → D3 tools → D4 Software Development → D5 real code execution → D6 tests/security evidence → D7 stabilization.

# 6. MEMBER 3 — MEMORY, TWINS, SPECIALISTS & PRODUCT

**Ownership:** Obsidian Memory, controlled research, five Executive Twins, Specialist registry/lifecycle and frontend/product integration. ■filecite■turn3file2■L99-L126■

## Memory APIs

| API | Purpose | Output |
|---|---|---|
| POST /memory/search | Retrieve Obsidian/company knowledge | MemoryResult[] |
| POST /memory/research | Controlled external research | sources + evidence |
| POST /memory/validate | Validate evidence before trust | approval decision |
| POST /memory/write | Write approved state/knowledge | noteId + status |
| GET /memory/context/{taskId} | Task context | context + sources |

## Executive Twin APIs

| API | Purpose | Output |
|---|---|---|
| POST /twins/resolve | Determine strategic need | twinId + reason |
| POST /twins/{id}/activate | Activate CEO/COO/CTO/CMO/CFO | activation state |
| POST /twins/{id}/recommend | Strategic recommendation | recommendation + rationale |
| POST /twins/{id}/delegate | Delegate concrete work | assignment |
| POST /twins/{id}/review | Review project result | review result |

## Specialist + Product APIs

| API | Purpose |
|---|---|
| GET /agents | Registry/filter specialists |
| POST /agents/spawn | Spawn worker |
| POST /agents/{id}/assign | Assign task/step |
| POST /agents/{id}/terminate | Terminate completed worker |
| GET /agents/{id} | Status/progress/tools/model |
| GET /events | Activity feed |
| WS /ws/tasks/{id} | Live task/workforce updates |

## Member 3 — 7-Day Delivery

D1 frontend/contracts → D2 task/status integration → D3 Obsidian Memory → D4 Twins + Specialist registry → D5 workforce/memory integration → D6 walkthrough/activity → D7 final integration.

## 7. Canonical API Lifecycles

**Basic:** /perception → /tasks → /capabilities/resolve → /security/check → /models/infer or /agents/spawn → /executions → /verifications → output → optional /memory/write.

**Advanced:** /perception → /projects → /tasks → /memory/search → /tasks/{id}/plan → /capabilities/resolve → /security/check → /twins/resolve if needed → /twins/{id}/activate → /agents/spawn → /executions → /verifications.

**Recovery:** verification failure → /reflections → /replans → security check → /tasks/{id}/retry → execution → verification.

**Completion:** verification pass → workers complete/terminate → artifacts → walkthrough → approved /memory/write.

## 8. Real-Time Event Contract

| Event | Producer | Consumers |
|---|---|---|
| task.created | Orchestrator | Frontend/logs |
| plan.created / plan.revised | Planner | Frontend/workforce |
| agent.spawned / assigned / running / completed / terminated | Agent Manager | Frontend/Orchestrator |
| tool.started / completed / failed | Execution | Security/UI |
| verification.started / passed / failed | Verification | Reflection/UI |
| reflection.created | Reflection | Re-plan Gate |
| replan.approved / rejected | Re-plan Gate | Planner/UI |
| memory.retrieved / written | Memory | Planner/UI |
| walkthrough.ready | Project service | Frontend |

## 9. Collaboration Rules

1) One repository. 2) Shared schemas. 3) Feature branches. 4) No direct cross-module database writes. 5) Every request carries taskId/correlation ID. 6) Every execution returns structured result + evidence. 7) Frontend consumes APIs/events, not internal module state. 8) New capabilities plug into Capability Manager + Specialist registry + model/tool adapters.

## 10. Reliability & Hallucination Controls

• **Evidence before re-plan:** tests, logs, stack traces, diffs and runtime output.

• **Re-plan Gate:** evidence support + scope + expected impact + verification criteria.

• **Independent verification:** the model generating a fix cannot be the sole authority declaring success.

• **Bounded retry:** recommended MVP maximum of 3 recovery attempts.

• **Memory safety:** external research is validated before becoming approved Obsidian knowledge.

## 11. Recommended 7-Day Implementation Stack

**Backend:** FastAPI + Pydantic. **Realtime:** WebSocket. **Execution isolation:** Docker. **AI:** local model service behind one Model Service interface. **Frontend:** existing Next.js application. These are implementation recommendations; the supplied source establishes the architecture and responsibilities rather than requiring these exact frameworks.

## 12. Day-7 API Definition of Done

✓ Basic Mode can enter, route, execute, verify and return a result.

✓ Advanced Project Mode can initialize a project and create a versioned plan.

✓ Memory retrieves from the supplied Obsidian knowledge base.

✓ Capability Manager resolves capability → worker/tool/model requirements.

✓ Guardrail returns allow/block/escalate with audit record.

✓ Executive Twin activates when strategic reasoning is required and delegates to specialists.

✓ Specialists can spawn, assign, run, verify, complete and terminate.

✓ Software Development creates/modifies a controlled project and runs tests.

✓ Verification produces objective evidence.

✓ Reflection/Re-plan operates from verification evidence.

✓ Retry is bounded and observable.

✓ Frontend receives real task/agent/project events.

✓ Approved results can be written to Obsidian.

✓ Typecheck, production build and core integration tests pass.